

Kiến trúc SOA & Microservices

Đặng Ngọc Hùng

Mục lục

5. Giao tiếp Bất đồng bộ — Kafka, Events & Messaging	4
5.1. Động lực của Giao tiếp Bất đồng bộ	5
5.1.1. Vấn đề mà sync không giải quyết được	5
5.1.2. Messaging patterns	6
5.2. Message Broker — Hai trường phái thiết kế	7
5.2.1. Durable vs Ephemeral Brokers	7
5.2.2. RabbitMQ — Smart Routing & Flexible Messaging	8
5.2.3. So sánh chi tiết	9
5.2.4. Khi nào dùng gì?	10
5.3. Apache Kafka — Kiến trúc chi tiết	11
5.3.1. Các khái niệm cốt lõi	11
5.4. Producer/Consumer Pattern trong KBM	12
5.4.1. Kafka Producer	12
5.4.2. Kafka Consumer	13
5.4.3. Flow hoàn chỉnh	14
5.5. Delivery Guarantees & Idempotency	15
5.5.1. Ba cấp độ đảm bảo	15
5.5.2. Idempotency — Thiết kế để chịu được duplicate	15
5.6. Event Schema Design	16
5.6.1. Cấu trúc một event	16
5.6.2. Bốn loại message	17
5.6.3. Schema evolution	18
5.6.4. Kafka vs DB Queue trong KBM	19
5.7. WebSocket — Real-time Notifications	19
5.7.1. Bài toán	19
5.8. Case Study: KBM	20
5.8.1. Bài toán Contest	20
5.8.2. Kiến trúc pipeline 2 Kafka topic	21
5.8.3. Phân tích các vấn đề	21
5.8.4. Đề xuất migration	22
5.9. Tổng kết	23
5.10. Đọc thêm	24
Tài liệu tham khảo	24

5.

CHƯƠNG 5.

Giao tiếp Bất đồng bộ – Kafka, Events & Messaging

“Event streams become the heart of data sharing throughout the company. Data no longer sits solely on a database accessible only through synchronous interfaces.”

— Hugo Rocha, *Practical Event-Driven Microservices Architecture* [5]

MỤC TIÊU HỌC TẬP

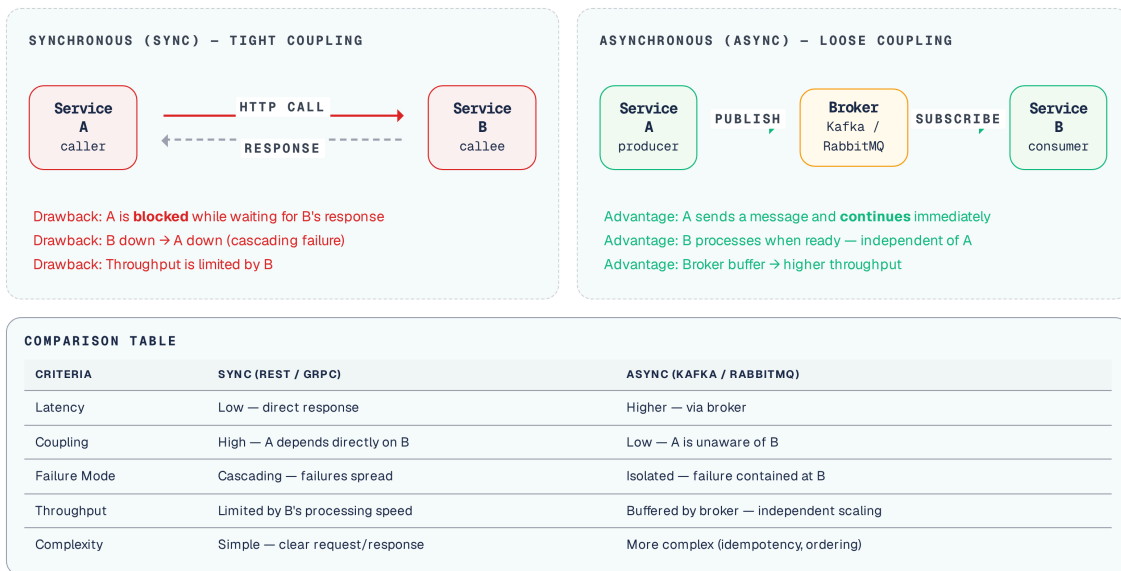
- Hiểu tại sao async messaging giải quyết các giới hạn của giao tiếp đồng bộ
- Phân biệt hai loại message broker: durable (Kafka) vs ephemeral (RabbitMQ)
- Nắm vững kiến trúc Apache Kafka: topics, partitions, consumer groups
- Hiểu kiến trúc RabbitMQ: exchanges, queues, bindings
- Thiết kế event schema: 4 loại message (Command, Event, Document, Query)
- So sánh delivery guarantees: at-most-once, at-least-once, exactly-once
- Sử dụng WebSocket cho real-time notifications
- Phân tích Kafka pipeline và DB queue trong KBM

5.1. Động lực của Giao tiếp Bất đồng bộ

Giống như việc sinh viên nộp đơn vào hồ sơ thay vì đứng chờ trực tiếp trước phòng giáo vụ, giao tiếp bất đồng bộ cho phép hệ thống tiếp nhận yêu cầu và xử lý sau, giải phóng các dịch vụ khỏi sự chờ đợi chéo nhau gây tắc nghẽn.

5.1.1. Vấn đề mà sync không giải quyết được

Chương 4 đã phân tích ba vấn đề cốt lõi của giao tiếp đồng bộ: sự phụ thuộc về mặt thời gian (temporal coupling), sự cố dây chuyền (cascading failures), và sự tích lũy độ trễ (latency accumulation). Giao tiếp bất đồng bộ (*asynchronous messaging*) giải quyết cả ba bằng cách **tách rời** (*decouple*) producer và consumer qua một trung gian — message broker.



Hình 5.1: So sánh giao tiếp đồng bộ (coupled) và bất đồng bộ (decoupled)

Thay vì bị khóa cứng về mặt thời gian, giao tiếp bất đồng bộ giải quyết dứt điểm từng điểm yếu này. Với bài toán **Temporal coupling**, producer chỉ cần gửi thông điệp vào broker rồi tiếp tục công việc khác, trong khi consumer có thể xử lý sau khi đã sẵn sàng. Tương tự, nó chặn đứng **Cascading failure** bởi nếu một service tạm ngừng hoạt động, thông điệp vẫn nằm an toàn tại broker chờ đến khi service đó phục hồi. Về mặt **Latency**, thay vì bắt người dùng phải chờ đợi hệ thống xử lý xong toàn bộ chuỗi tác vụ, ứng dụng có thể trả về phản hồi ngay lập tức và thông báo kết quả thực tế qua cơ chế notification hoặc polling sau đó.

💡 Broker thay vì Async HTTP

Nhiều lập trình viên mới thắc mắc: “**Tại sao không gửi một HTTP request và không cần chờ kết quả (fire-and-forget)?**”

Câu trả lời là: Giao tiếp HTTP bất đồng bộ giống như một cuộc gọi điện thoại — dù người gọi truyền đạt thông tin rất nhanh rồi ngắt kết nối, người nhận vẫn bắt buộc phải đang trong trạng thái sẵn sàng tiếp nhận (online) ngay tại thời điểm đó. Còn Message Broker giống như việc **sinh viên nộp đơn phúc khảo vào hộp thư Phòng Đào tạo**. Người gửi bỏ đơn vào hộp (Broker), hệ thống cam kết sẽ giữ đơn và chuyển đến tay chuyên viên, kể cả khi máy chủ người nhận đang gặp sự cố hay mất điện/sự cố hạ tầng.

Richardson trong [2a, Ch.3] phân tích: async messaging cải thiện **tính sẵn sàng (availability)** vì các dịch vụ không phụ thuộc lẫn nhau tại runtime. Tuy nhiên, hệ thống phải đánh đổi bằng **độ phức tạp (complexity)** — tính nhất quán cuối cùng (eventual consistency), thứ tự thông điệp (message ordering) và xử lý trùng lặp (duplicate handling) — đây là những chi phí không thể tránh khỏi.

Việc thay đổi phương thức giao tiếp đòi hỏi hệ thống phải sẵn sàng chấp nhận những độ trễ nhất định trong việc đồng bộ trạng thái. Dựa trên nền tảng đó, chúng ta cần tuân thủ nghiêm ngặt các quy tắc để tránh hiện tượng ràng buộc ngàm giữa các dịch vụ.

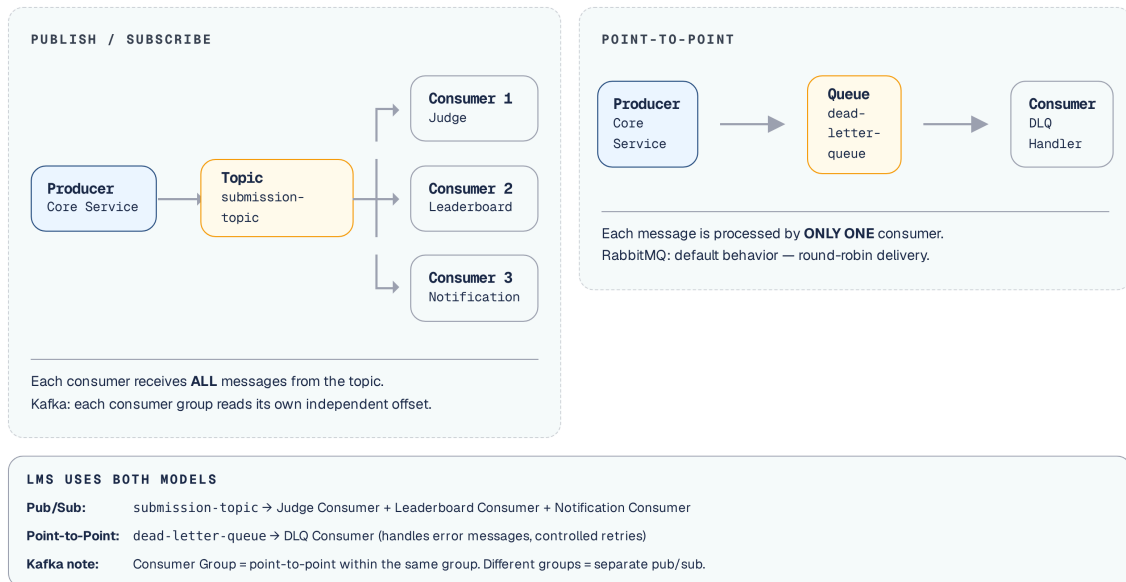
⚠️ Đổi Consistency lấy Availability

Async messaging cải thiện tính sẵn sàng và giảm temporal coupling — các dịch vụ không cần online cùng lúc — nhưng phải trả giá bằng tính nhất quán cuối cùng, khử trùng lặp (deduplication) và xử lý dead letter. Đây là sự đánh đổi có chủ đích, không phải hệ quả ngẫu nhiên. Trước khi chọn async, câu hỏi bắt buộc: “Khoảng thời gian chờ nhất quán (consistency window) bao lâu là chấp nhận được cho bài toán thực tế này?”

5.1.2. Messaging patterns

Có ba messaging patterns cơ bản [2a, Ch.3]:

1. **Point-to-point (Queue)** — Một message, một consumer. Giống như giảng viên giao đề tài riêng cho một sinh viên trên hệ thống LMS (sinh viên này đã nhận và xử lý thì người khác không cần làm nữa). Phù hợp cho command: “hãy chấm bài này”.
2. **Publish/Subscribe (Topic)** — Một message, nhiều consumer. Giống như Phòng đào tạo gửi thông báo toàn trường (bất kỳ phòng ban nào quan tâm đều tiếp nhận được thông tin nguyên bản độc lập với nhau). Phù hợp cho event: “bài đã được chấm xong”.
3. **Request/Async Response** — Gửi request qua messaging, nhận response qua message khác (có correlation ID). Kết hợp lợi ích của cả hai.



Hình 5.2: Point-to-Point (Queue) vs Publish/Subscribe (Topic)

 Command vs Event

Phân biệt rõ **command** (yêu cầu hành động: “ChấmBài”, “GửiEmail”) và **event** (thông báo sự kiện đã xảy ra: “BàiĐãĐượcChấm”, “ĐiểmĐãCậpNhật”). Các lệnh (commands) thường được phân phối theo mô hình point-to-point, trong khi các sự kiện (events) thường tuân theo mô hình pub/sub. Sự phân biệt này ảnh hưởng đến thiết kế schema và error handling [5, Ch.8].

5.2. Message Broker — Hai trường phái thiết kế

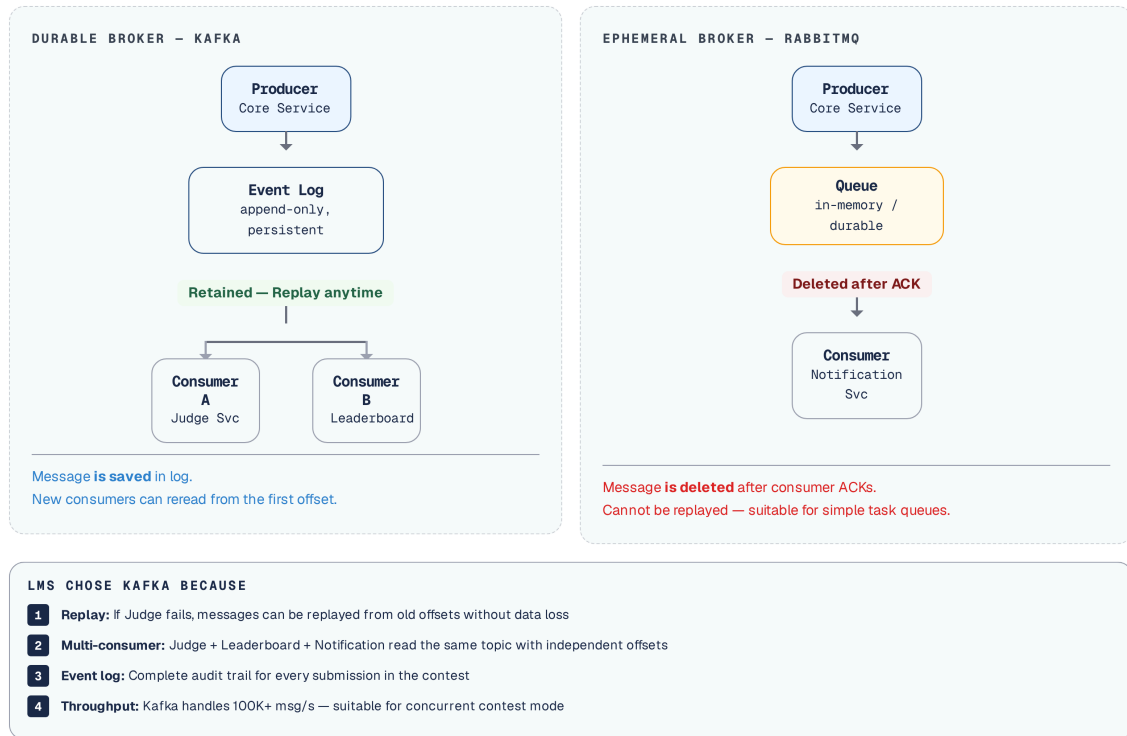
Message Broker hoạt động giống như trung tâm bưu điện của trường đại học. Tùy thuộc vào cách thức lưu trữ và phân phát thư tín, các thiết kế được chia thành hai trường phái trái ngược, quyết định trực tiếp đến sức chịu tải của toàn bộ hệ thống.

5.2.1. Durable vs Ephemeral Brokers

Rocha trong [5, §3.1.3] phân loại message brokers thành hai trường phái cơ bản dựa trên vòng đời của message:

Ephemeral (tạm thời) – Thông điệp bị xóa sau khi consumer gửi phản hồi xác nhận (acknowledge).
Đại diện: **RabbitMQ**, **ActiveMQ**, **Amazon SQS**.

Durable (bền vững) – Thông điệp được lưu trữ trên ổ cứng (disk) và vẫn sẵn sàng (available) sau khi được tiêu thụ (consume). Đại diện: **Apache Kafka**, **Apache Pulsar**, **Amazon Kinesis**. Hệ sinh thái này cũng xuất hiện các lựa chọn mới đáng chú ý: **Redpanda** (tương thích Kafka API nhưng viết bằng C++, một binary duy nhất, không cần JVM/ZooKeeper — gọn nhẹ cho self-hosted) và **NATS JetStream** (nhẹ, độ trễ thấp, phù hợp hệ thống nhỏ). Nội dung chương dùng Kafka làm đại diện; các khái niệm partition, offset, consumer group áp dụng gần như nguyên vẹn cho Redpanda.



Hình 5.3: Ephemeral broker (message xóa sau ack) vs Durable broker (message vẫn còn)

Ở đây, “ephemeral” chỉ việc message bị xóa sau khi được xác nhận tiêu thụ (destructive consumption). Thuật ngữ này không có nghĩa RabbitMQ thiếu độ bền: durable queue và persistent message vẫn sống sót qua lần khởi động lại. Ngược lại, “durable” theo nghĩa của Kafka là message được giữ lại theo retention policy để đọc lại (replay), chứ không mặc nhiên vĩnh viễn.

Sự khác biệt này **không chỉ là kỹ thuật** — nó ảnh hưởng đến cách thiết kế hệ thống. Các broker lưu trữ bền vững (durable broker) cho phép phát lại (replay), lưu vết sự kiện (event sourcing), và hỗ trợ nhiều bên tiêu thụ (consumer) cùng đọc một luồng dữ liệu. Trong khi đó, các broker trung chuyển (ephemeral broker) tập trung vào định tuyến thông minh (smart routing), hàng đợi ưu tiên (priority queue), và giao tiếp điểm-điểm (point-to-point communication).

Chọn Broker theo Use Case

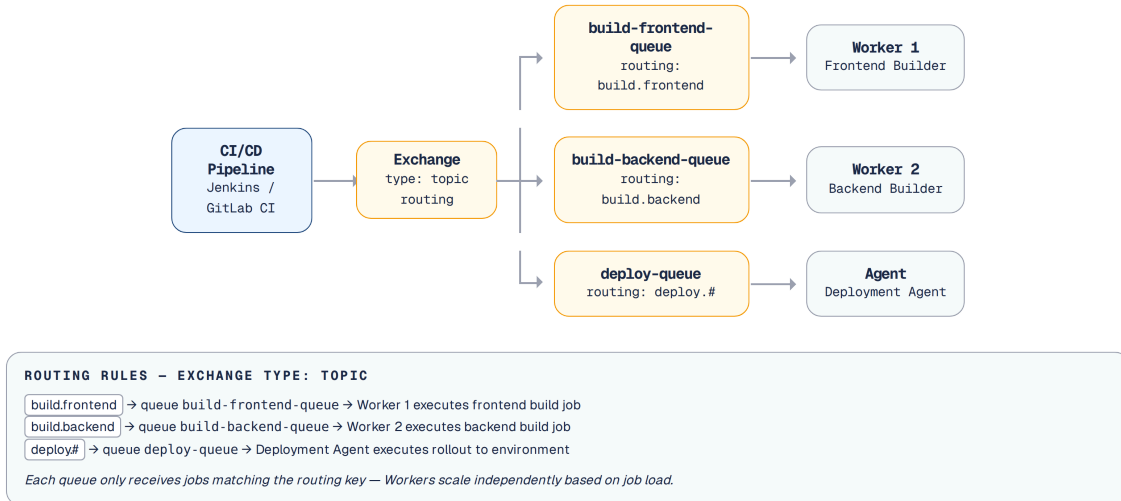
Durable brokers (Kafka) phù hợp khi cần replay, event sourcing, và multiple consumers. Ephemeral brokers (RabbitMQ) phù hợp khi cần smart routing và point-to-point delivery. Chọn broker dựa trên **thói quen của đội ngũ phát triển** thay vì đặc điểm kỹ thuật sẽ dẫn đến sự lệch pha (mismatch) kiến trúc khó sửa chữa về sau.

Việc lựa chọn broker giống như việc thiết lập kênh thông báo trong trường đại học. Nếu cần lưu trữ toàn bộ lịch sử điểm số để sinh viên có thể tra cứu lại bất cứ lúc nào (event sourcing), Kafka là hệ thống số cái đào tạo phù hợp nhất. Ngược lại, nếu chỉ cần phân luồng đơn từ phúc khảo đến đúng phòng ban xử lý và có thể bỏ đi sau khi giải quyết xong, RabbitMQ hoạt động như một hòm thư tiếp nhận thông minh.

5.2.2. RabbitMQ — Smart Routing & Flexible Messaging

RabbitMQ dựa trên **AMQP** (Advanced Message Queuing Protocol), với kiến trúc phong phú hơn Kafka về routing:

Trong môi trường đại học, không phải mọi thông báo đều được phát loa toàn trường. RabbitMQ cung cấp một cơ chế định tuyến thông minh (smart routing), giống như một nhân viên phân loại thư tín tại Phòng Hành chính, đảm bảo mỗi đơn từ hay yêu cầu nghiệp vụ đều được chuyển phát chính xác đến tay người cần xử lý.



Hình 5.4: Kiến trúc RabbitMQ — Exchange routing messages đến queues

Để khai thác tối đa ưu điểm của nền tảng này, chúng ta cần nắm vững các thành phần kiến trúc cốt lõi cấu thành nên hệ sinh thái của nó. Khác với mô hình append-only log đơn giản, luồng dữ liệu trong RabbitMQ phải đi qua một trạm trung chuyển thông minh trước khi thực sự hạ cánh xuống nơi lưu trữ.

Dưới đây là các khái niệm cốt lõi của RabbitMQ:

Exchange – Nhận message và định tuyến (route) đến queues theo các quy tắc thiết lập trước.

Queue – Lưu trữ message trong khi chờ consumer tiêu thụ (hoạt động theo nguyên tắc FIFO).

Binding – Quy tắc kết nối từ exchange đến queue (thông qua routing key, headers).

Ack/Nack – Cơ chế để Consumer xác nhận đã xử lý xong (ack) hoặc từ chối (nack → requeue/dead letter).

Exchange types – Direct, Fanout, Topic, Headers — mỗi loại cung cấp một logic định tuyến riêng biệt. Trong khi Kafka chú trọng vào khả năng lưu trữ bền vững và thông lượng cực lớn bằng cách phân vùng dữ liệu đơn giản theo partition, thì RabbitMQ lại có thể mạnh ở năng lực **smart routing**. Một thông điệp trong RabbitMQ có thể được nhân bản, lọc hoặc định tuyến phức tạp đến đúng các hàng đợi mục tiêu dựa trên routing key, cấu trúc header matching, hoặc broadcast tới nhiều thành phần của hệ thống.

5.2.3. So sánh chi tiết

Dưới đây là so sánh chi tiết giữa hai hệ thống này theo từng tiêu chí cụ thể:

Loại hệ thống – Kafka là một durable event log, lưu trữ lâu dài. RabbitMQ là ephemeral message queue, xóa thông điệp sau khi xử lý.

Mô hình dữ liệu – Kafka sử dụng cấu trúc append-only log, nơi consumer theo dõi qua offset. RabbitMQ sử dụng hàng đợi FIFO (Queue FIFO), thông điệp biến mất sau khi được xác nhận (ack).

Throughput (Thông lượng) – Kafka đạt mức rất cao (hàng triệu thông điệp mỗi giây). RabbitMQ ở mức trung bình (hàng chục nghìn thông điệp mỗi giây).

Đảm bảo thứ tự (Ordering) – Kafka đảm bảo thứ tự trên từng partition. RabbitMQ đảm bảo thứ tự trong cùng một hàng đợi.

Khả năng Replay – Kafka hỗ trợ chạy lại (replay) từ bất kỳ offset nào. RabbitMQ mặc định không thể replay (trừ khi dùng RabbitMQ Streams bản 3.9+).

Cơ chế Định tuyến (Routing) – Kafka định tuyến đơn giản qua topic và partition key. RabbitMQ định tuyến linh hoạt bằng 4 loại exchange và routing keys.

Hàng đợi ưu tiên (Priority) – Kafka không hỗ trợ mức ưu tiên. RabbitMQ hỗ trợ tốt Priority queues.

Hẹn giờ thông điệp (Delayed messages) – Kafka không có tính năng nguyên bản. RabbitMQ giải quyết qua Delayed message exchange.

Giao thức (Protocol) – Kafka dùng custom binary protocol tối ưu riêng. RabbitMQ hỗ trợ đa dạng (AMQP, STOMP, MQTT).

Trường hợp sử dụng (Use case) – Kafka dành cho event sourcing, stream processing, audit log (ví dụ: lịch sử thao tác của sinh viên). RabbitMQ phù hợp cho task queues, delayed jobs, complex routing (ví dụ: phân luồng các lệnh in ấn bảng điểm, gửi email).

Sự khác biệt này quyết định môi trường áp dụng của mỗi broker. Việc chọn sai công cụ cho bài toán sẽ nhanh chóng tạo ra các nút thắt cổ chai.

RabbitMQ trong High-throughput

Rocha chia sẻ kinh nghiệm từ một e-commerce platform lớn [5, §3.1.3]: Nhóm phát triển (team) đã sử dụng RabbitMQ nhiều năm trong môi trường thực tế có thông lượng cao (production high-throughput). Vấn đề: Khi tải trọng đạt đỉnh (peak load) dẫn đến hiện tượng tích tụ thông điệp, **toàn bộ cụm máy chủ (cluster) bị ảnh hưởng** – các dịch vụ không liên quan cũng bị chậm. Nhìn lại, kịch bản sử dụng thực tế của họ (event streaming, high throughput) phù hợp với broker lưu trữ bền vững (durable broker) hơn. Bài học: **chọn broker phải dựa trên bài toán thực tế**, không phải theo thói quen.

Ví dụ minh họa thực tiễn: Để các worker thực sự chia đều công việc (mô hình Competing Consumers), team cấu hình `prefetch` nhỏ (ví dụ `prefetch=1`) cho các tác vụ chăm bài kéo dài. `prefetch` giới hạn số message chưa ACK được giao trước cho mỗi consumer: không có giới hạn này, RabbitMQ dồn giao trước (round-robin) hàng loạt message cho worker đang bận trong khi worker khác đã rảnh. Đổi lại, `prefetch` quá nhỏ làm giảm thông lượng với tác vụ ngắn – đây là tham số tuning theo độ dài tác vụ, không phải hằng số bất biến.

5.2.4. Khi nào dùng gì?

Nhu cầu	Chọn
Event sourcing, audit trail, replay	Kafka
High throughput (100K+ msg/sec)	Kafka
Stream processing (KStreams, ksqlDB)	Kafka
Task/job queues, background workers	RabbitMQ
Complex routing (headers, topic patterns)	RabbitMQ
Priority queues, delayed messages	RabbitMQ
Cả hai	Kafka cho event backbone + RabbitMQ cho task queues

Bảng 5.1: Khi nào chọn Kafka, khi nào chọn RabbitMQ

KBM chọn Kafka cho pipeline xử lý bài nộp (submissions) vì cần khả năng phát lại (**replay**) — cho phép chấm lại bài khi logic hệ thống thay đổi, đảm bảo thứ tự (ordering) theo người dùng, và hỗ trợ khả năng mở rộng khi một kỳ thi (contest) có nhiều lượt nộp bài cùng lúc [5, Ch.3]. Tuy nhiên, Kafka không phải câu trả lời mặc định cho mọi queue: DevOps Lab dùng **DB queue** — bảng database với status field (**PENDING** , **PROCESSING** , **DONE**) và worker poll định kỳ — cho job vận hành vì cần transactional enqueue đơn giản, dễ quan sát cùng dữ liệu domain và không cần replay event log toàn hệ thống.

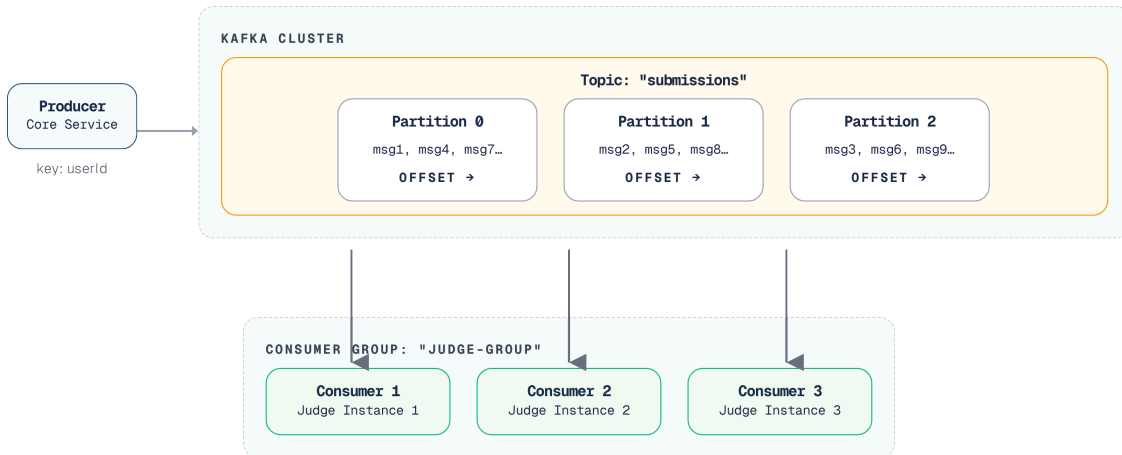
❗ KBM: Hai Công Nghệ, Hai Use Case Khác Nhau

KBM sử dụng Kafka cho luồng xử lý bài nộp (submission pipeline) và hàng đợi cơ sở dữ liệu cho các tác vụ của DevOps Lab — đây là quyết định đúng đắn về khả năng phát lại (replayability) và thông lượng (throughput). Tuy nhiên, việc áp dụng Kafka cho submission (một long-running task) cũng mang lại rủi ro Head-of-Line blocking nghiêm trọng, buộc đội ngũ phải triển khai thêm các workaround phức tạp (như Thread Pool concurrency) sẽ được thảo luận ở phần sau. Job vận hành cần transactional enqueue, dễ monitor bằng SQL, và không cần scale lớn → hàng đợi cơ sở dữ liệu là đủ và đơn giản hơn nhiều. **Việc chuẩn hóa cục đoạn (over-standardizing) trên Kafka** cho mọi hàng đợi sẽ tạo ra gánh nặng vận hành (operational overhead) không cần thiết.

5.3. Apache Kafka — Kiến trúc chi tiết

Tương tự như việc phân chia các khoa và lớp học để quản lý hàng ngàn sinh viên, hệ sinh thái Kafka cũng tổ chức luồng dữ liệu thành các cấu trúc phân cấp nhằm tối đa hóa khả năng xử lý song song.

5.3.1. Các khái niệm cốt lõi



Hình 5.5: Kiến trúc Kafka — Topic, Partitions và Consumer Group

Hệ sinh thái Kafka được cấu thành từ các khái niệm nền tảng sau:

Concept	Mô tả	Ví dụ KBM
Topic	Luồng message theo category	<code>submissions</code> , <code>judge-results</code> , <code>notifications</code>
Partition	Chia topic thành segments song song	1 partition cho <code>submissions</code> (hiện tại); đề xuất tăng lên 3 để scale
Consumer Group	Nhóm các consumer chia nhau các partition	<code>judge-group</code> — 3 Judge instances (hiện chỉ 1 active do <code>submissions</code> có 1 partition)
Offset	Vị trí đọc tiếp theo mà consumer đã commit — như một cái Dấu kẹp sách (Bookmark) đặt <i>ngay sau</i> message cuối cùng đã xử lý thành công	Consumer 1 xử lý xong message offset 42 → commit position 43; sau restart đọc tiếp từ offset 43
Retention	Thời gian giữ message	7 ngày (default) hoặc vĩnh viễn

Bảng 5.2: Các concepts cốt lõi của Apache Kafka

Partition key quyết định message vào partition nào. Các thông điệp có cùng khóa (key) sẽ được phân bổ vào cùng một phân vùng (partition), qua đó đảm bảo được thứ tự xử lý. Trong KBM, partition key = `userId` hoặc key theo bài/contest giúp đảm bảo ordering ở đúng phạm vi nghiệp vụ.

Kleppmann trong [7, Ch.11] giải thích: Kafka kết hợp ưu điểm của database (durable storage, replay) với ưu điểm của messaging (real-time, decoupling) — đây là mô hình **log-based messaging** khác biệt cơ bản với queue-based messaging.

5.4. Producer/Consumer Pattern trong KBM

Trong hệ thống LMS, Producer hoạt động như cổng tiếp nhận bài thi của sinh viên để đẩy vào đường ống xử lý. Ngược lại, Consumer giống như hệ thống phân ban, âm thầm lấy bài thi từ đường ống ra để chấm điểm một cách độc lập và tuần tự.

5.4.1. Kafka Producer

Trong KBM, submissions được gửi qua Kafka khi sinh viên nộp bài:

```
// Producer: Core Service gửi submission vào Kafka
@Service
public class SubmitProducer extends BaseProducerService<SubmitMessage> {

    private static final String TOPIC = "submissions";

    public void sendSubmission(Submission submission) {
        SubmitMessage message = SubmitMessage.builder()
            .submissionId(submission.getId())
            .questionId(submission.getQuestionId())
            .userId(submission.getUserId())
            .sqlContent(submission.getSqlContent())
            .databaseType(submission.getDatabaseType())
            .build();

        // key = userId → đảm bảo ordering per user
        kafkaTemplate.send(TOPIC, submission.getUserId().toString(), message);
    }
}
```

Listing 5.1: Kafka Producer — Core Service gửi submission

5.4.2. Kafka Consumer

(Từ phần này, chương phân tích sâu các “bẫy” vận hành Kafka trong production — Head-of-Line Blocking, idempotency, poison pill, partition. Người đọc mới chỉ cần nắm: consumer đọc message từ topic theo group và xử lý. Các nội dung chuyên sâu về rủi ro vận hành bên dưới có thể được tham khảo lại ở các giai đoạn sau, khi đọc giả trực tiếp triển khai hệ thống thực tế.)

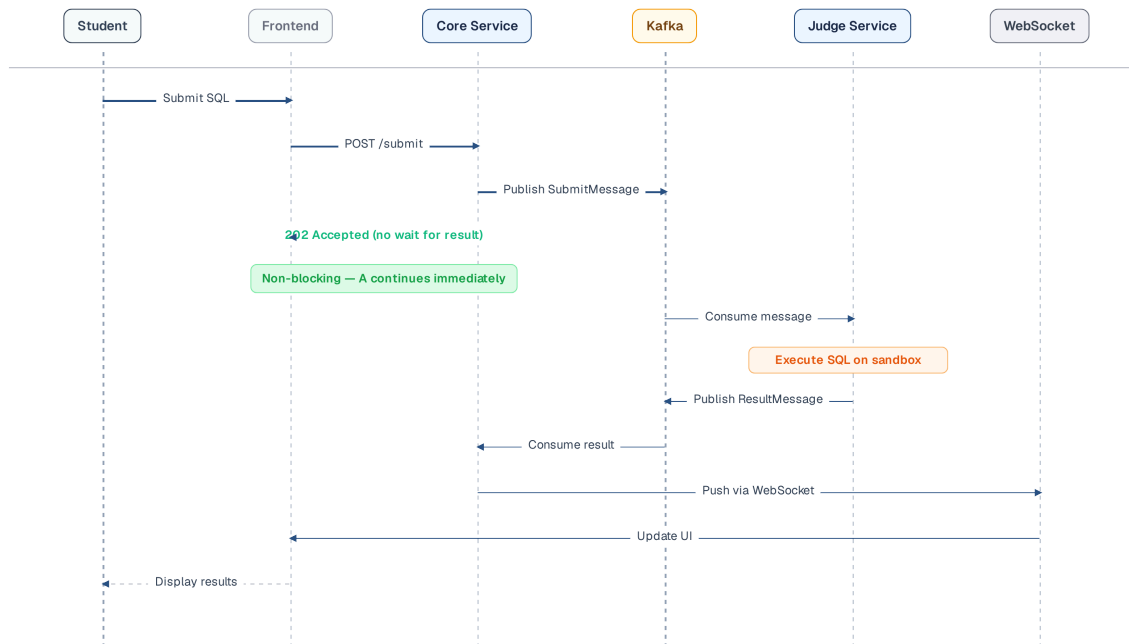
Judge Service consume submissions và trả kết quả:

```
// Consumer: Judge Service nhận và xử lý (Naive, Blocking)
@KafkaListener(
    topics = "submissions",
    groupId = "judge-group",
    containerFactory = "kafkaListenerContainerFactory"
)
public void processSubmission(SubmitMessage message) {
    // #sym.excl CẢNH BÁO THIẾT KẾ (ANTI-PATTERN): Việc gọi execute() đồng bộ ở đây sẽ chặn
    // thread của Kafka consumer. Nếu task SQL Judge chờ quá lâu và vi phạm cấu hình
    // max.poll.interval.ms, Kafka broker sẽ ghi nhận rằng consumer đã ngừng hoạt động (timeout) và kích hoạt
    // lỗi rebalance loop (Rebalance Storm), gây gián đoạn toàn hệ thống.
    JudgeResult result = sqlExecutorService.execute(message);

    // Gửi kết quả ngược lại qua topic khác
    resultProducer.send("judge-results", message.getSubmissionId(), result);
}
```

Listing 5.2: Kafka Consumer — naive synchronous implementation gây Head-of-Line Blocking

5.4.3. Flow hoàn chỉnh



Hình 5.6: Luồng hoàn chỉnh — từ submit qua Kafka đến kết quả qua WebSocket

Lưu ý: response là **202 Accepted** (không phải 200 OK) — nghĩa là “request đã nhận, đang xử lý”. User nhận kết quả qua WebSocket notification, không phải HTTP response.

❗ Thiếu error handling trong Kafka pipeline

Một số pipeline của KBM vẫn cần được gia cố (hardening) để đối phó với tình trạng lỗi xử lý thông điệp (message failures): nếu dịch vụ chấm điểm (Judge Service) ngừng hoạt động đột ngột (crash), thông điệp có thể bị mất hoặc bị xử lý lặp tùy thuộc vào cơ chế ghi nhận trạng thái đọc (commit offset). **Best practice** theo [2a, Ch.3]: sử dụng cơ chế ghi nhận độ lệch thủ công (manual offset commit) kết hợp với chủ đề dành riêng cho thông điệp lỗi (dead letter topic) đối với các giao dịch thất bại. **Lộ trình chuyển đổi (Migration path)**: (1) chuyển sang xác nhận thủ công (manual commit), (2) áp dụng `@RetryableTopic` cùng hàng đợi thông điệp lỗi (dead letter queue), (3) triển khai giám sát độ trễ của bên tiêu thụ (consumer lag monitoring).

Hãy tưởng tượng hàng trăm sinh viên nộp bài thi cùng lúc nhưng giảng viên lại phải chấm từng bài một theo thứ tự nghiêm ngặt; một bài thi quá khó sẽ làm đình trệ toàn bộ những người nộp sau.

⚠ Head-of-Line Blocking trong Kafka

Cần phân biệt rõ: dùng Kafka để *nhận* và *đệm* submission (enqueue nhanh, throughput cao, replay) là hợp lý — đó chính là lý do KBM chọn Kafka. Vấn đề chỉ phát sinh khi *xử lý đồng bộ* tác vụ chậm dài (hàng chục giây) *ngay trên consumer thread*: lúc đó nó trở thành anti-pattern nghiêm trọng. Kafka hoạt động theo cơ chế partition, một thông điệp (message) xử lý chậm sẽ chặn toàn bộ các thông điệp phía sau trong cùng partition (hiện tượng **Head-of-Line Blocking**). Với các tác vụ chạy dài (long-running task) không yêu cầu thứ tự nghiêm ngặt (strict ordering), mô hình Competing Consumers của **RabbitMQ** tỏ ra hiệu quả hơn. Nếu bắt buộc phải dùng Kafka, hệ thống cần triển khai Thread Pool phía sau consumer để tránh nghẽn. (Lưu ý: Nếu tự xây dựng Thread Pool, việc quản lý manual offset commit sẽ rất phức tạp vì các task hoàn thành không theo thứ tự, dễ gây mất dữ liệu khi service crash. Do đó, nên ưu tiên dùng các thư viện chuyên dụng đã xử lý sẵn bài toán này như **Confluent Parallel Consumer**).

Lưu ý (Rebalance Storm): Khi consumer xử lý một batch các long-running task, tổng thời gian rất dễ vượt quá giới hạn `max.poll.interval.ms` (mặc định là 5 phút). Khi đó, Kafka broker sẽ ghi nhận rằng consumer đã ngừng hoạt động (timeout) và kích hoạt rebalance, trong khi consumer thực tế vẫn đang xử lý. Điều này dẫn đến vòng lặp rebalance vô tận (Endless Rebalance). Để khắc phục, cần tuning cấu hình: tăng `max.poll.interval.ms` hoặc giảm `max.poll.records` (số lượng message mỗi lần poll) để đảm bảo thời gian xử lý batch luôn nằm trong ngưỡng an toàn.

5.5. Delivery Guarantees & Idempotency

Quản lý thông điệp cũng giống như việc xác nhận sinh viên nộp học phí. Hệ thống cần cơ chế kiểm soát chặt chẽ để cam kết dữ liệu được ghi nhận chính xác, đảm bảo không ai bị trừ tiền hai lần cho cùng một hóa đơn đóng học.

5.5.1. Ba cấp độ đảm bảo

Đây là một trong những khái niệm quan trọng nhất trong messaging — và cũng dễ bị hiểu sai nhất [7, Ch.11]:

Trong hệ thống phân tán, các cơ chế giao nhận thông điệp được chia thành ba cấp độ chính: **At-most-once** (gửi tối đa một lần, ưu tiên tốc độ nhưng rủi ro mất mát dữ liệu); **At-least-once** (gửi ít nhất một lần, đảm bảo độ tin cậy nhưng có thể tạo ra các thông điệp trùng lặp đòi hỏi cơ chế xử lý idempotent); và **Exactly-once** (xử lý đúng một lần, là cấp độ cao nhất nhưng đòi hỏi chi phí hiệu năng rất đắt đỏ).

Kleppmann trong [7, Ch.11] phân tích: **exactly-once semantics** trong thực tế là *effectively-once* — đạt được bằng cách kết hợp at-least-once delivery + idempotent processing. Kafka transactions (since 0.11) hỗ trợ exactly-once **trong Kafka** nhưng không across systems (Kafka → DB).

5.5.2. Idempotency — Thiết kế để chịu được duplicate

Khi sử dụng chiến lược at-least-once (phổ biến nhất), phía tiêu thụ (consumer) *có thể* nhận cùng một thông điệp (message) nhiều lần. Mã nguồn phải đảm bảo idempotency — xử lý nhiều lần cho cùng kết quả [5, Ch.8]:


```

// #sym.times Không idempotent — chấm trùng = điểm sai
@KafkaListener(topics = "submissions")
public void process(SubmitMessage msg) {
    JudgeResult result = judge(msg);
    submissionRepository.updateScore(msg.getSubmissionId(), result.getScore());
    // Nếu message trùng → score bị cộng dồn hoặc ghi đè không đúng state
}

// #sym.checkmark Idempotent — check trước khi xử lý
@KafkaListener(topics = "submissions")
public void process(SubmitMessage msg) {
    if (submissionRepository.isAlreadyJudged(msg.getSubmissionId())) {
        log.info("Submission {} already judged, skipping", msg.getSubmissionId());
        return; // Skip duplicate
    }
    JudgeResult result = judge(msg);
    submissionRepository.updateScore(msg.getSubmissionId(), result.getScore());
}

```

Listing 5.3: Idempotent consumer — kiểm tra trước khi xử lý

Rocha trong [5, Ch.8] nhấn mạnh: event consumption phải **idempotent** — xử lý nhiều lần vẫn cho cùng kết quả — và, khi domain cho phép, nên thiết kế thêm tính commutative/associative để chịu được thứ tự xáo trộn. Với các luồng bất buộc đúng thứ tự, hệ thống nên dựa vào ordering theo partition key (mục trên) và version/sequence của aggregate để từ chối hoặc trì hoãn event sai thứ tự, thay vì cố ép mọi phép xử lý phải có tính giao hoán.

Race Window trong Check-then-write

Mẫu `isAlreadyJudged()` rồi `updateScore()` ở trên đủ để minh họa ý tưởng, nhưng *chưa* an toàn tuyệt đối khi hai bản sao consumer xử lý cùng một message song song: cả hai có thể cùng vượt qua bước kiểm tra trước khi bất kỳ bên nào kịp ghi. Để idempotency chắc chắn, cần dựa vào **ràng buộc ở tầng database** — ví dụ **UNIQUE constraint** trên `submissionId`, thao tác **UPSERT / INSERT ... ON CONFLICT DO NOTHING**, hoặc một bảng “đã xử lý” (processed-message table) trong cùng transaction với việc ghi kết quả.

Trong KBM, idempotency đặc biệt quan trọng ở SQL Judge: submission có thể được retry qua Kafka/queue, và `judge` coordinator có thể gọi lại cùng một `judge-*` worker sau timeout. Mỗi command nên có `submissionId` hoặc `judgeRunId` ổn định; consumer/worker kiểm tra trạng thái trước khi ghi kết quả. Nếu đã chấm xong, hệ thống trả về kết quả hiện có hoặc bỏ qua message, thay vì tạo thêm một lần chấm mới.

5.6. Event Schema Design

Mọi thông báo gửi đi trong trường học đều cần tuân thủ một biểu mẫu chuẩn. Trong kiến trúc sự kiện, việc thống nhất cấu trúc dữ liệu chung này là yêu cầu bắt buộc, giúp các dịch vụ đọc hiểu nhau dễ dàng mà không vấp phải lỗi tương thích.

5.6.1. Cấu trúc một event

Rocha đề xuất mỗi event nên có ba phần [5, Ch.8]:

```
{
  "metadata": {
    "eventId": "e7d3f2a1-...",
    "eventType": "SubmissionJudged",
    "version": "1.0",
    "timestamp": "2026-03-10T14:30:00Z",
    "source": "kbm-judge-sql",
    "correlationId": "req-abc-123"
  },
  "payload": {
    "submissionId": "sub-456",
    "questionId": "q-789",
    "userId": "user-001",
    "result": "CORRECT",
    "executionTimeMs": 245
  }
}
```

Listing 5.4: Cấu trúc event với metadata và payload

Một event tiêu chuẩn cần tách biệt rõ ràng giữa hai phần: **metadata** làm nhiệm vụ cung cấp các siêu dữ liệu hỗ trợ hệ thống theo dõi (tracing), chống trùng lặp (deduplication) và quản lý phiên bản (versioning) qua các trường như `eventId` hay `correlationId`; trong khi đó, **payload** chứa các dữ liệu nghiệp vụ (business data) cụ thể của domain đó. Trong hệ sinh thái Cloud Native, cấu trúc tách biệt giữa metadata và payload này đã được chuẩn hóa thành tiêu chuẩn công nghiệp **CloudEvents** (do CNCF quản lý). Việc tuân thủ cấu trúc tương tự CloudEvents giúp hệ thống dễ tích hợp hơn với hệ sinh thái công cụ hỗ trợ chuẩn này.

5.6.2. Bốn loại message

Rocha phân loại message thành 4 loại, mỗi loại có mục đích và naming convention riêng [5, §3.1.4]:

Loại	Mô tả	Naming	Ví dụ KBM	Delivery
Command	Yêu cầu thực hiện hành động – <i>có thể bị từ chối</i>	Verb imperative	<code>JudgeSubmission</code> , <code>SendNotification</code>	Point-to-point
Event	Thông báo sự kiện đã xảy ra – <i>sự thật, không thể reject</i>	Past participle	<code>SubmissionJudged</code> , <code>ContestStarted</code>	Pub/sub
Document	Snapshot toàn bộ entity khi thay đổi – <i>full state, hiện thân của Event-Carried State Transfer Pattern</i>	Noun	<code>SubmissionDocument</code> , <code>UserDocument</code>	Pub/sub
Query	Yêu cầu thông tin, không thay đổi state	Noun + request	Thường qua HTTP (broker có thể dùng Request-Reply Pattern nhưng ít phổ biến)	Sync (hoặc RPC)

Bảng 5.3: Bốn loại message trong event-driven architecture

5.6.3. Schema evolution

Quá trình tiến hóa cấu trúc dữ liệu (Schema evolution) diễn ra tương tự như việc nâng cấp API versioning đã đề cập ở Chương 3. Việc thay đổi mẫu biểu sự kiện (event schema) cũng đòi hỏi một chiến lược tiến hóa nhất quán [7, Ch.4]. Hãy hình dung biểu mẫu đăng ký học phần của sinh viên qua các năm:

Thêm field optional (Tương thích ngược) – Bổ sung một cột “Nguyện vọng 2” không bắt buộc. Biểu mẫu cũ vẫn dùng được, biểu mẫu mới mang thêm thông tin. Điều này là an toàn (✓).

Bỏ field (Phá vỡ cấu trúc) – Xóa bỏ cột “Mã số sinh viên”. Các hệ thống (consumer) cũ vẫn phụ thuộc vào thông tin này để xử lý hồ sơ, dẫn đến lỗi toàn hệ thống (×).

Đổi field type (Phá vỡ cấu trúc) – Thay đổi mã môn học từ dạng số sang chuỗi ký tự dài. Các hệ thống cũ vốn lưu mã môn là số nguyên sẽ lập tức bị gián đoạn (×).

Thực tiễn tốt nhất (Best practice) là triển khai một **Schema Registry** (như Confluent Schema Registry hoặc AWS Glue) hoạt động như một kho lưu trữ khuôn mẫu trung tâm. Giống như việc Phòng Đào tạo phải kiểm duyệt và đóng dấu mọi thay đổi trên mẫu đơn trước khi phát hành, mỗi sự thay đổi về schema đều được hệ thống tự động kiểm chứng mức độ tương thích trước khi cho phép ban hành (publish).

i KBM thiếu event schema chuẩn

Các thông điệp trong KBM là plain Java objects (POJOs) được tuần tự hóa (serialize) dưới định dạng JSON — không có metadata chuẩn (eventId, timestamp, correlationId), không có schema registry, không có version. Khi Judge Service thay đổi định dạng thông điệp (format message), Core Service có thể gặp sự cố (crash) nếu chưa được cập nhật. **Migration path:** (1) thêm metadata wrapper cho tất cả Kafka messages, (2) sử dụng `eventId` cho idempotency checks, (3) cân nhắc Avro + Schema Registry khi quy mô đội ngũ phát triển mở rộng.

5.6.4. Kafka vs DB Queue trong KBM

Use case	Lựa chọn	Lý do
SQL submission / contest	Kafka	Cần throughput, ordering theo key, replay và consumer group
SQL Judge worker dispatch (<code>judge</code> → <code>judge-*</code>)	Kafka hoặc DB queue + worker	Mỗi tác vụ (job) mang một định danh (<code>submissionId</code> / <code>judgeRunId</code>) ổn định, thuận tiện cho việc thử lại (retry) và đảm bảo idempotency; các trình thực thi (worker) của DBMS sẽ xử lý độc lập
DevOps Lab operations	DB queue	Job gắn chặt với state trong database, cần transaction enqueue cùng thay đổi domain
Audit/learning analytics	Kafka	Nhiều consumers có thể đọc cùng event stream

Bảng 5.4: Chọn Kafka hay DB queue theo use case

Việc sử dụng DB queue không làm giảm đi tính chất microservices của hệ thống so với Kafka. Nó là lựa chọn hợp lý khi lưu lượng (volume) vừa phải, số lượng phía tiêu thụ (consumer) ít, và yêu cầu quan trọng nhất là tính toàn vẹn (atomicity) giữa trạng thái nghiệp vụ và bản ghi tác vụ. Kafka tỏ ra vượt trội hơn khi hệ thống cần phân tán thông điệp (fan-out), phát lại dữ liệu (replay), phân vùng (partitioning) và xử lý luồng (stream processing).

5.7. WebSocket — Real-time Notifications

Thay vì để sinh viên phải liên tục làm mới trang web để xem điểm thi, hệ thống cần một kênh liên lạc chủ động. WebSocket mở ra một kết nối hai chiều, cho phép bảng điểm tự động nhảy số ngay khi giảng viên hoàn tất việc chấm bài.

5.7.1. Bài toán

Sau khi Judge Service chấm xong, kết quả cần đến tay sinh viên. Có ba cách:

Để cập nhật trạng thái cho người dùng, hệ thống có ba phương án tiếp cận: **Polling** (client liên tục hỏi định kỳ, dễ cài đặt nhưng rất lãng phí băng thông và độ trễ cao); **Long polling** (client giữ kết nối chờ server có dữ liệu mới trả lời, tuy khắc phục được độ trễ nhưng lại khó mở rộng); và cuối cùng là **WebSocket** (kết

nổi hai chiều cho phép server chủ động đẩy dữ liệu thời gian thực, đánh đổi lại là chi phí quản lý trạng thái kết nối).

KBM sử dụng **STOMP over WebSocket** — sub-protocol messaging chạy trên WebSocket, triển khai bằng Spring WebSocket (có thể bật thêm SockJS fallback qua `.withSockJS()` khi cần hỗ trợ trình duyệt cũ; khi đó client phải dùng `websocketFactory` thay cho `brokerURL`):

```
// Server-side: push kết quả chấm bài
@Service
public class NotificationService {
    private final SimpMessagingTemplate messagingTemplate;

    public void notifyJudgeResult(UUID userId, JudgeResult result) {
        messagingTemplate.convertAndSendToUser(
            userId.toString(),
            "/queue/notifications",
            new JudgeNotification(result)
        );
    }
}
```

Listing 5.5: WebSocket server — push kết quả chấm bài qua STOMP

```
// Client-side: subscribe nhận kết quả
const stompClient = new StompJs.Client({
    brokerURL: 'ws://<gateway-host>/ws'
});

stompClient.onConnect = () => {
    stompClient.subscribe('/user/queue/notifications', (message) => {
        const result = JSON.parse(message.body);
        showNotification(`Kết quả: ${result.status} #sym.checkmark`);
    });
};
```

Listing 5.6: WebSocket client — subscribe nhận kết quả real-time

WebSocket phù hợp cho **notification cuối pipeline** (kết quả chấm bài, cập nhật leaderboard trong contest) nhưng không thay thế Kafka cho **inter-service messaging** — hai vai trò khác nhau.

5.8. Case Study: KBM

Kỳ thi tính giờ trên KBM là thử thách khắc nghiệt về hiệu năng. Hàng trăm sinh viên đồng loạt nộp bài sẽ tạo ra áp lực khổng lồ, đòi hỏi đường ống dữ liệu phải vận hành ổn định như một cỗ máy thu bài tự động tại phòng thi.

5.8.1. Bài toán Contest

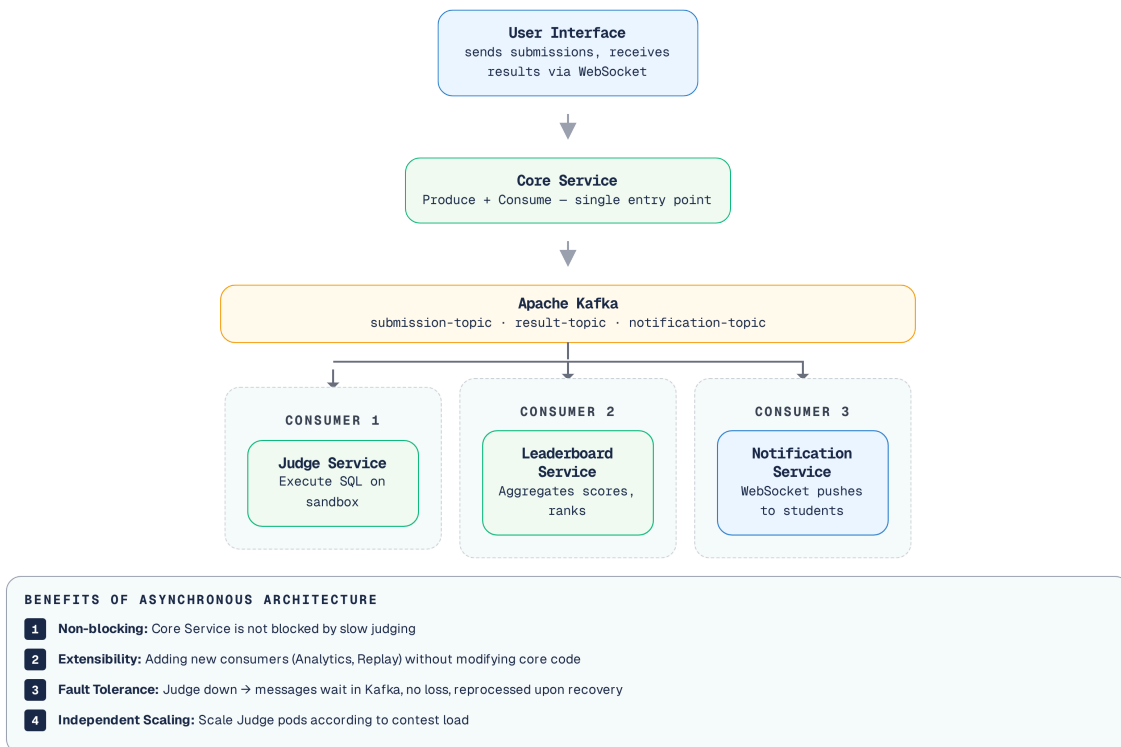
Trong Contest mode, 100+ sinh viên đồng thời nộp bài trong thời gian giới hạn (60–120 phút). Yêu cầu:

- High throughput: Xử lý hàng trăm lượt nộp bài (submissions) mỗi phút
- Fair ordering: Đảm bảo thứ tự xử lý cho từng user
- Real-time feedback: Sinh viên thấy kết quả ngay
- Leaderboard update: Bảng xếp hạng cập nhật liên tục

⚠ **Cạm bẫy: key co-location**

Yêu cầu “fair ordering” (partition theo `userId`) lại có mặt trái: khi nhiều user rơi vào cùng một partition (key co-location — hệ quả bình thường của partition theo key, không phải hash collision), bài nộp chậm của người này chặn người kia. Sinh viên A nộp một bài giải có vòng lặp vô hạn mất 30 giây để xử lý sẽ chặn đứng bài nộp chỉ cần 1 giây của sinh viên B nếu cả hai thông điệp rơi vào cùng một phân vùng (partition). Đây chính là Head-of-Line Blocking đã phân tích ở trên.

5.8.2. Kiến trúc pipeline 2 Kafka topic



Hình 5.7: Kiến trúc pipeline 2 Kafka topic chấm bài trong Contest mode

Để đáp ứng các yêu cầu khắt khe trên, đội ngũ KBM đã thiết kế một đường ống luân chuyển dữ liệu dựa trên Kafka. Luồng xử lý này chia nhỏ toàn bộ vòng đời của một lượt nộp bài thành nhiều chặng riêng biệt, mỗi chặng được quản lý bởi một topic chuyên dụng.

Topic	Producer	Consumer	Message	Key
<code>submissions</code>	Core Service	Judge Service	SQL + metadata	<code>userId</code>
<code>judge-results</code>	Judge Service	Core Service	Result + timing	<code>submissionId</code>
(WebSocket)	Core Service	Frontend	Notification	<code>userId</code>

Bảng 5.5: Chi tiết các Kafka topics trong pipeline

5.8.3. Phân tích các vấn đề

Trong thực tế, luồng thiết kế này khi chịu tải cao đã bộc lộ một số vấn đề về cấu hình và xử lý exception.

#	Vấn đề	Hiện trạng	Best Practice [2a]
1	Auto-commit offset	Commit trước khi xử lý xong → message loss	Manual commit sau khi xử lý thành công
2	Không có DLT	Thông điệp lỗi bị thử lại (retry) vô hạn hoặc bị mất	Cấu hình Dead Letter Topic cho các thông điệp lỗi
3	Không idempotent	Thông điệp trùng lặp (duplicate) có thể tạo ra kết quả sai	Kiểm tra <code>submissionId</code> trước khi chấm điểm (judge)
4	Thiếu monitoring	Không nắm được độ trễ của phía tiêu thụ (consumer lag)	Thiết lập Kafka consumer lag metrics và alerting
5	Single partition	Tất cả bài nộp (submissions) bị đẩy vào 1 phân vùng dẫn đến tắc nghẽn (bottleneck)	Tăng partitions, thiết lập key = <code>userId</code>

Bảng 5.6: Phân tích vấn đề Kafka pipeline trong KBM

5.8.4. Đề xuất migration

Quá trình chuyển đổi và củng cố pipeline cần được chia thành các giai đoạn rõ ràng. Trong **Phase 1 — Reliability** (ưu tiên cao), hệ thống cần đảm bảo không mất mát dữ liệu bằng cách sử dụng manual offset commit kết hợp `@RetryableTopic` (với 3 lần thử lại) và cấu hình Dead Letter Topic. Đồng thời, phải bổ sung idempotency check thông qua `submissionId`.

Chuyển sang **Phase 2 — Observability** (cũng ưu tiên cao), trọng tâm là khả năng giám sát. Hệ thống cần triển khai Kafka consumer lag monitoring sử dụng Kafka Exporter và Prometheus để theo dõi độ trễ. Việc thêm `correlationId` xuyên suốt pipeline (từ submission đến judge, result, và notification) cũng bắt buộc để dễ dàng truy vết (tracing).

Cuối cùng, ở **Phase 3 — Performance** (khi cần mở rộng quy mô), hệ thống bắt buộc phải tách biệt (decouple) Consumer Thread và Execution Thread. Consumer chỉ nên chịu trách nhiệm lấy dữ liệu rồi đẩy vào Local Thread Pool để xử lý thực tế (kèm cơ chế quản lý offset cẩn trọng để tránh mất dữ liệu do out-of-order execution), hoặc ưu tiên sử dụng thư viện chuyên dụng như Confluent Parallel Consumer. Việc chỉ tăng số lượng partition (ví dụ lên 3) sẽ không giải quyết được tình trạng tắc nghẽn nếu có nhiều bài nộp chạy chậm cùng lúc. Bên cạnh đó, việc thêm các Judge instances vào consumer group cũng cần thiết để tăng thông lượng xử lý — với điều kiện số partition được tăng tương ứng: số consumer hoạt động đồng thời trong một group không thể vượt quá số partition của topic, instance dư ra sẽ chỉ đứng chờ (idle).

⚠ Sai lầm thường gặp khi làm việc với Message Broker

Khi triển khai các hệ thống hàng đợi trong thực tế, các nhóm phát triển thường vấp phải một số cái bẫy kinh điển, dẫn đến mất mát dữ liệu hoặc sập hệ thống dưới tải trọng cao. Việc nhận diện và phòng tránh những sai lầm này là bắt buộc để duy trì tính ổn định của hệ thống chấm điểm hay bất kỳ hệ thống phân tán nào.

Dưới đây là các sai lầm phổ biến và cách phòng tránh:

Auto-commit offset trước khi xử lý xong – Phía tiêu thụ (consumer) xác nhận (acknowledge) thông điệp trước khi quá trình xử lý (processing) hoàn thành. Giống như việc chuyên viên đào tạo đóng dấu “đã xử lý” lên đơn đăng ký tin chỉ trước khi thực sự cập nhật vào hệ thống; nếu hệ thống tắt nguồn đột

ngột lúc đó, đơn sẽ bị quên lãng vĩnh viễn. *Phòng tránh*: Dùng manual offset commit — chỉ commit *sau* khi processing thành công.

Không có Dead Letter Topic – Thông điệp lỗi bị thử lại (retry) vô hạn hoặc làm tắc nghẽn (block) toàn bộ pipeline. Một đơn xin bảo lưu bị lỗi định dạng có thể chặn đứng mọi đơn từ khác xếp sau nó (hiện tượng poison pill). *Phòng tránh*: Cấu hình DLT (`@RetryableTopic` trong Spring Kafka) — chuyển các message lỗi sau N lần thử sang một hàng đợi riêng để xử lý thủ công.

Không thiết kế idempotent consumer – Giả định mỗi message chỉ đến đúng một lần. Khi Kafka rebalance, một bài tập có thể bị chấm hai lần, khiến sinh viên nhận được hai kết quả khác nhau. *Phòng tránh*: Kiểm tra trạng thái trước khi xử lý, nếu đã chấm rồi thì bỏ qua (§5.5).

Chọn broker vì quen thuộc, không vì use case – Sử dụng RabbitMQ cho luồng sự kiện (event streaming) chỉ vì đội ngũ phát triển đã quen thuộc với công nghệ này. Khi lượng lớn sinh viên truy cập cùng lúc trong kỳ thi, broker không phù hợp sẽ dẫn đến quá tải. *Phòng tránh*: Đánh giá use case trước (§5.2) — chọn Kafka cho event streaming, RabbitMQ cho task queues.

Interactive Demo

Xem minh họa động về **Cơ chế hoạt động của Message Broker** tại companion web: message-broker.html

5.9. Tổng kết

Giao tiếp bất đồng bộ giải quyết ba giới hạn cốt lõi của sync: temporal coupling, cascading failures, và latency. Hai trường phái message broker — durable (Kafka) và ephemeral (RabbitMQ) — phục vụ use case khác nhau, và nhiều hệ thống dùng cả hai.

Kafka với mô hình event log (durable, replayable, high-throughput) trở thành nền tảng phổ biến nhất cho event-driven architecture. RabbitMQ với smart routing và flexible messaging phù hợp cho task queues, delayed jobs, và complex routing patterns. Partitions, consumer groups, và exchange types cho phép mỗi broker scale theo cách riêng.

Delivery guarantees (at-most/at-least/exactly-once) và idempotency là hai khái niệm không thể tách rời. Thiết kế event schema chuẩn — với metadata, versioning, và correlation — là đầu tư cần thiết giúp hệ thống dễ dàng gỡ lỗi (debug) và tiến hóa (evolve).

Phân tích KBM cho thấy pipeline Kafka hoạt động nhưng thiếu nhiều best practice quan trọng: error handling, idempotency, monitoring. Đồng thời, DevOps Lab nhắc lại một bài học thực tế: có những job queue nên nằm gần database thay vì ép qua Kafka. Đây là nợ kỹ thuật (technical debt) và sự đánh đổi (trade-off) phổ biến khi các đội ngũ nhỏ ưu tiên tính năng trước độ tin cậy (reliability).

Ở Chương 6, chúng ta sẽ đối mặt với bài toán khó nhất của distributed systems: **distributed transactions**. Khi data nằm ở nhiều service khác nhau, làm thế nào để đảm bảo consistency? Saga pattern sẽ là câu trả lời.

Bài tập Chương 5

Xem Phụ lục Bài tập để luyện tập:

- 5-1** – Kafka vs RabbitMQ — Quyết định dựa trên gì? (Trade-off Analysis [L2])
- 5-2** – Debugging: “Messages biến mất” — Dead Letter & Poison Pills ([L3])

5.10. Đọc thêm

Sách tham khảo chính:

1. [5] Hugo Rocha, *Practical Event-Driven MS Architecture* — Ch.1: Why EDA; Ch.3: Kafka; Ch.8: Event Schema Design
2. [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. — Ch.3: Async Messaging, Transactional Outbox
3. [7] Martin Kleppmann, *Designing Data-Intensive Applications* — Ch.11: Stream Processing, Event Logs

Sách bổ trợ:

4. [3] Mitra & Nadareishvili, *Microservices: Up and Running* — Ch.4: Event-Driven Communication

Nguồn trực tuyến:

- Apache Kafka documentation — kafka.apache.org/documentation
- Confluent Schema Registry — docs.confluent.io/platform/current/schema-registry
- Spring Kafka reference — docs.spring.io/spring-kafka

Tài liệu tham khảo

- Rocha, H. (2022). *Practical Event-Driven Microservices Architecture*. Apress.